SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Name : K.Mithreyi Reddy

Reg. No : 11249a221

Class : II B.E. (CSE)

Section :s3

Course Code :

Course Name :

SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that this is the bonafide record of work done by **Mr/Ms. K.Mithreyi Reddy** ,

with **Reg. No _11249a221**_____of **II-B.E.(CSE)** during the academic year 2025 – 2026.

Station:Enatur

Date:

Staff-in-charge **Head of the Department**

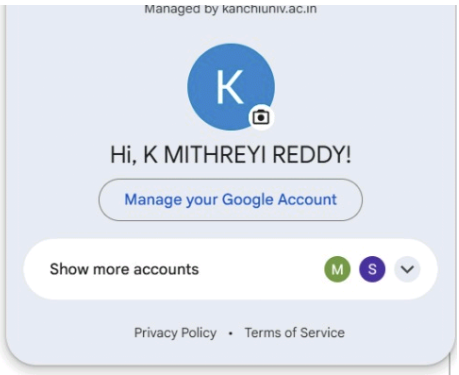
Submitted for the Practical examination held on_____.

Examiner-1 **Examiner-2**

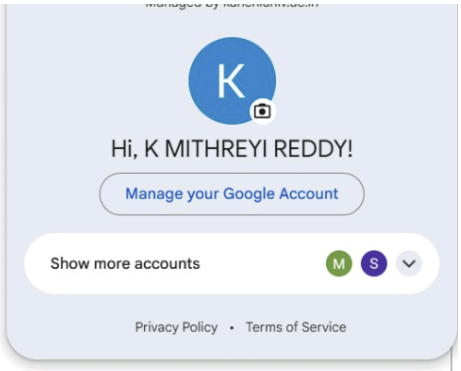
INDEX

```python
import numpy as np
# Create a list and convert to NumPy array
list1 = [1, 2, 3]
vector1 = np.array(list1)
print("Vector 1:", vector1)
# Create a range array
vector = np.arange(1, 6)
print("Arange vector:", vector)
# Create zeros array
vector = np.zeros(5)
print("Zeros:", vector)
# Dot product example
list1 = [5, 1, 1]
list2 = [3, 2, 1]
vector1 = np.array(list1)
vector2 = np.array(list2)
print("Vector 1:", vector1)
print("Vector 2:", vector2)
product = vector1.dot(vector2)
print("Dot product:", product)
# More NumPy examples
a = np.array([1, 2, 3, 4, 5])
b = np.arange(1, 10, 2)
c = np.linspace(0, 1, 5)
d = np.zeros((2, 3))
e = np.ones((2, 2))
f = np.eye(3)
g = np.random.randint(1, 100, size=(3, 3))
print("a:", a)
```

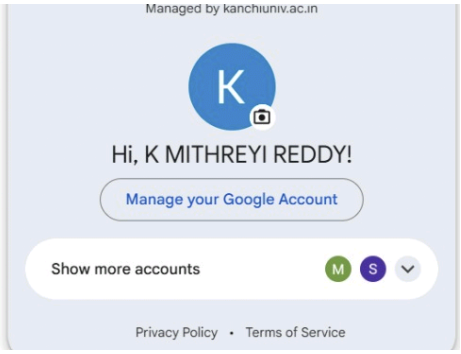
```
print("b:", b)
print("c:", c)
print("d:\n", d)
print("e:\n", e)
print("f (identity matrix):\n", f)
print("g (random):\n", g)
#2)Array properties
print("Shape:", g.shape)
print("Dimensions:", g.ndim)
print("Size:", g.size)
print("Data type:", g.dtype)
#3) Reshape & Transpose
reshaped = g.reshape(1, 9) # Changed 12 to 9 as g is 3x3
print("Reshaped array:\n", reshaped)
print("Transpose:\n", g.T)
print("Flattened:", g.flatten())
#4) Mathematical Operations
print("Add:", np.add(a, 2))
print("Multiply:", np.multiply(a, 3))
print("Square root:", np.sqrt(a))
print("Power:", np.power(a, 2))
print("Absolute:", np.abs([-2, 3]))
#5) Statistical Functions
print("Mean:", np.mean(a))
print("Median:", np.median(a))
print("Standard Deviation:", np.std(a))
print("Variance:", np.var(a))
print("Minimum:", np.min(a))
print("Maximum:", np.max(a))
```



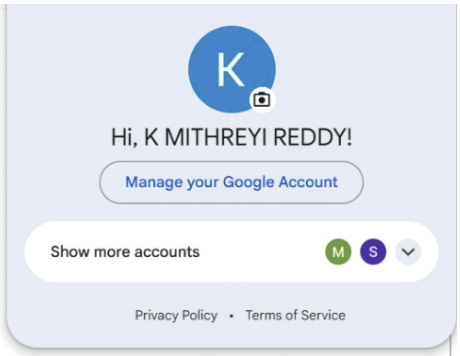
```
print("Sum:", np.sum(a))
print("Percentile (50):", np.percentile(a, 50))
#6)Axis-based operations
print("Column-wise sum:\n", np.sum(g, axis=0))
print("Row-wise mean:\n", np.mean(g, axis=1))
#7) Indexing & Slicing
print("First element:", a[0])
print("Slice:", a[1:4])
print("2D Slice:\n", g[0:2, 1:3])
#8)Boolean & Conditional Operations
print("Where > 50:\n", np.where(g > 50))
print("Any > 90:", np.any(g > 90))
print("All > 0:", np.all(g > 0))
#9)Searching + Sorting
print("Sorted a:", np.sort(a))
print("Index of max:", np.argmax(a))
print("Index of min:", np.argmin(a))
print("Unique elements:", np.unique([1, 2, 2, 3, 4, 4]))
#10)
x = np.array([[1, 2],[3, 4]])
y = np.array([[5, 6],[7, 8]])
print("Dot Product:\n", np.dot(x, y))
print("Matrix Multiplication:\n", np.matmul(x, y))
print("Determinant:", np.linalg.det(x))
print("Inverse:\n", np.linalg.inv(x))
#11)
import numpy as np
np.random.seed(1)
a = np.array([1, 2, 3, 4, 5])
```



```
rand_sample = np.random.choice(a, size=3)
print("Random choice:", rand_sample)
#12)# Create arrays
h1 = np.array([1, 2, 3])
h2 = np.array([4, 5, 6])
# Concatenate and Stack
print("Concatenate:", np.concatenate((h1, h2)))
print("Vertical Stack:\n", np.vstack((h1, h2)))
print("Horizontal Stack:", np.hstack((h1, h2)))
# Removed duplicate import numpy as np
#13)# Save array to file
np.save("sample-array.npy", a)
# Load array from file
loaded = np.load("sample-array.npy")
print("Loaded array:", loaded)
```

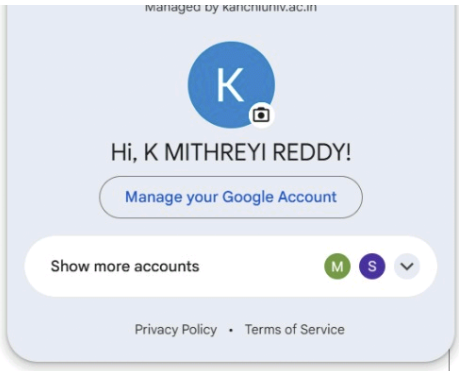


```
Vector 1: [1 2 3]
Arange vector: [1 2 3 4 5]
Zeros: [0. 0. 0. 0. 0.]
Vector 1: [5 1 1]
Vector 2: [3 2 1]
Dot product: 18
a: [1 2 3 4 5]
b: [1 3 5 7 9]
c: [0.   0.25 0.5  0.75 1.  ]
d:
 [[0. 0. 0.]
 [0. 0. 0.]]
e:
 [[1. 1.]
 [1. 1.]]
f (identity matrix):
 [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
g (random):
 [[ 3 91 75]
 [13  3 31]
 [75 75 95]]
Shape: (3, 3)
Dimensions: 2
Size: 9
Data type: int64
Reshaped array:
 [[ 3 91 75 13  3 31 75 75 95]]
```

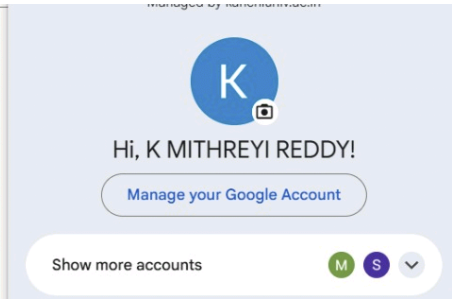


```
Transpose:
 [[ 3 13 75]
 [91  3 75]
 [75 31 95]]
Flattened: [ 3 91 75 13  3 31 75 75 95]
Add: [3 4 5 6 7]
Multiply: [ 3  6  9 12 15]
Square root: [1.         1.41421356 1.73205081 2.         2.23606798]
Power: [ 1  4  9 16 25]
Absolute: [2 3]
Mean: 3.0
Median: 3.0
Standard Deviation: 1.4142135623730951
Variance: 2.0
Minimum: 1
Maximum: 5
Sum: 15
Percentile (50): 3.0
Column-wise sum:
 [ 91 169 201]
Row-wise mean:
 [56.33333333 15.66666667 81.66666667]
First element: 1
Slice: [2 3 4]
2D Slice:
 [[91 75]
 [ 3 31]]
Where > 50:
 (array([0, 0, 2, 2, 2]), array([1, 2, 0, 1, 2]))
Any > 90: True
All > 0: True
Sorted a: [1 2 3 4 5]
```

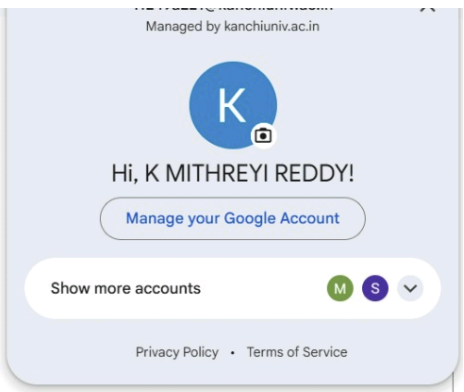
```
Index of max: 4
Index of min: 0
Unique elements: [1 2 3 4]
Dot Product:
 [[19 22]
 [43 50]]
Matrix Multiplication:
 [[19 22]
 [43 50]]
Determinant: -2.0000000000000004
Inverse:
 [[-2.   1. ]
 [ 1.5 -0.5]]
Random choice: [4 5 1]
Concatenate: [1 2 3 4 5 6]
Vertical Stack:
 [[1 2 3]
 [4 5 6]]
Horizontal Stack: [1 2 3 4 5 6]
Loaded array: [1 2 3 4 5]
```



```python
import pandas as pd
data = {
"Name": ["Alice", "Bob", "Charlie"],
"Age": [25, 30, 35],
"City": ["New York", "Paris", "London"]
}

df = pd.DataFrame(data)


print(df.head())
print(df.info())
print(df.describe())
```



```
      Name  Age      City
0    Alice   25  New York
1      Bob   30     Paris
2  Charlie   35    London
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    3 non-null      object
 1   Age     3 non-null      int64
 2   City    3 non-null      object
dtypes: int64(1), object(2)
memory usage: 204.0+ bytes
None
        Age
count   3.0
mean   30.0
std     5.0
min    25.0
25%    27.5
50%    30.0
75%    32.5
max    35.0
```



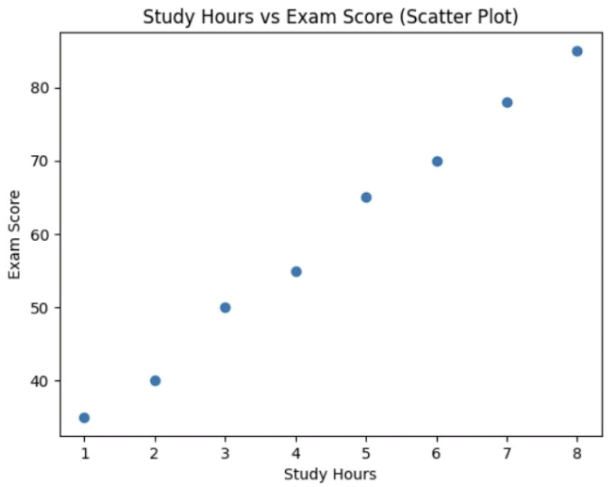
```python
ages = df["Age"]
above_25 = df[df["Age"] > 25]
row_0 = df.iloc[0]

# Specific value (row 0, City column)
specific_val = df.loc[0, "City"]

print(ages)
print(above_25)
print(row_0)
print(specific_val)
```

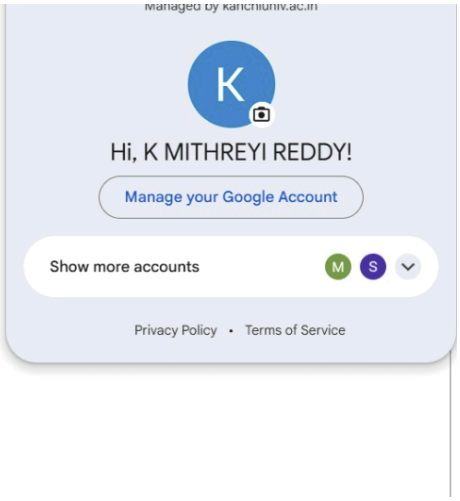
```
0    25
1    30
2    35
Name: Age, dtype: int64
      Name  Age    City
1      Bob   30   Paris
2  Charlie   35  London
Name       Alice
Age           25
City    New York
Name: 0, dtype: object
New York
```

```python
print(df.isnull().sum())
df_clean = df.dropna()
df_filled = df.fillna(0)
# Fill missing Age values with mean
df["Age"] = df["Age"].fillna(df["Age"].mean())
print(df_clean)
print(df_filled)
```

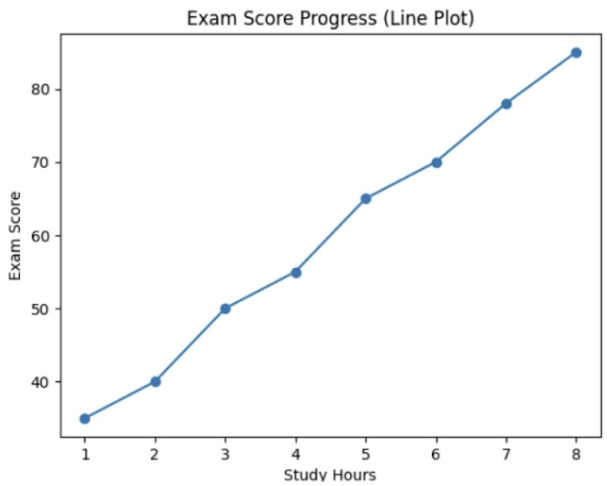
```
Name    0
Age     0
City    0
dtype: int64
      Name  Age      City
0    Alice   25  New York
1      Bob   30     Paris
2  Charlie   35    London
      Name  Age      City
0    Alice   25  New York
1      Bob   30     Paris
2  Charlie   35    London
```

```python
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
data = {
"StudyHours": [1, 2, 3, 4, 5, 6, 7, 8],
"ExamScore": [35, 40, 50, 55, 65, 70, 78, 85]
}
df = pd.DataFrame(data)
plt.scatter(df["StudyHours"], df["ExamScore"])
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Study Hours vs Exam Score (Scatter Plot)")
plt.show()
```

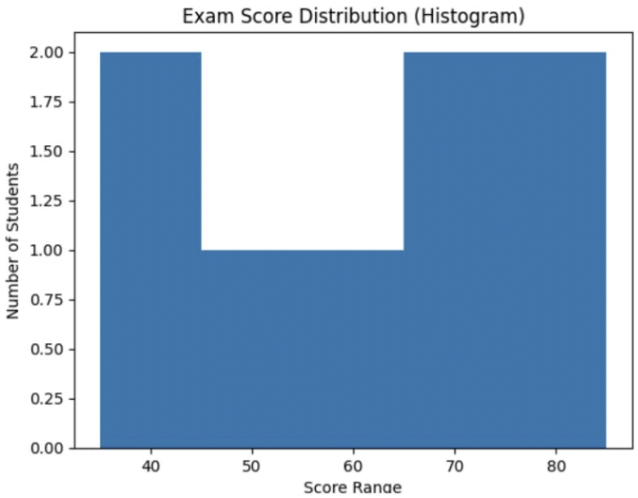

Study Hours vs Exam Score (Scatter Plot)
Exam Score
Study Hours
80
70
60
50
40
1
2
3
4
5
6
7
8


Managed by kanchiuniv.ac.in
K
Hi, K MITHREYI REDDY!
Manage your Google Account
Show more accounts
M
S
Privacy Policy • Terms of Service

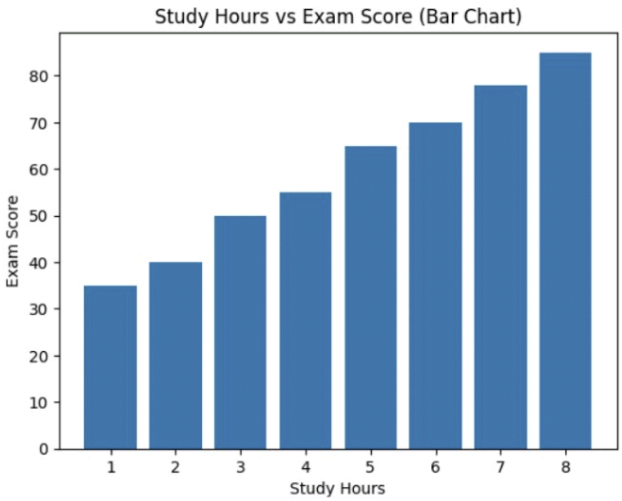
```
plt.plot(df["StudyHours"], df["ExamScore"], marker='o')
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Exam Score Progress (Line Plot)")
plt.show()
```



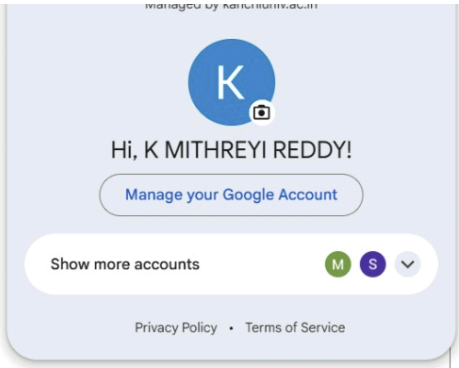
```
plt.hist(df["ExamScore"], bins=5)
plt.xlabel("Score Range")
plt.ylabel("Number of Students")
plt.title("Exam Score Distribution (Histogram)")
plt.show()
```



```
plt.bar(df["StudyHours"], df["ExamScore"])
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Study Hours vs Exam Score (Bar Chart)")
plt.show()
```

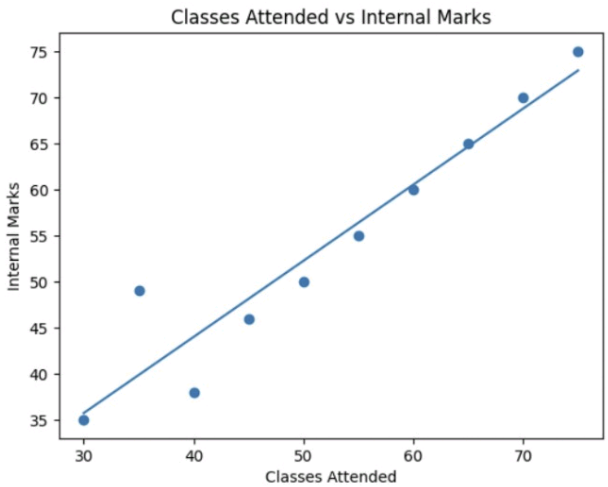


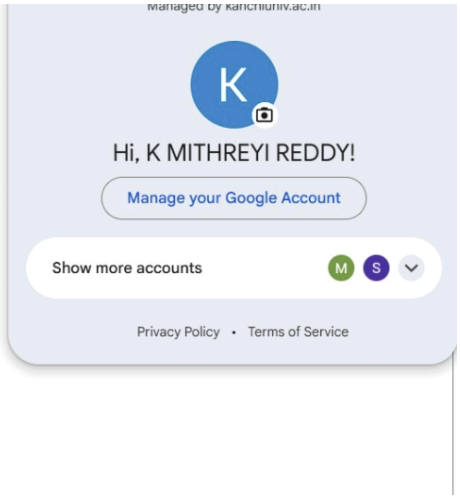
```python
# STEP 1: IMPORT LIBRARIES
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
# STEP 2: CREATE DATASET
data = {
"Student": ["S1", "S2", "S3", "S4", "S5",
"S6", "S7", "S8", "S9", "S10"],
"Classes_Attended": [30, 35, 40, 45, 50,
55, 60, 65, 70, 75],
"Internal_Marks": [35, 49, 38, 46, 50,
55, 60, 65, 70, 75]
}
df = pd.DataFrame(data)
print(df)
# STEP 3: FEATURES + TARGET
X = df[["Classes_Attended"]] # Feature
y = df["Internal_Marks"] # Target
# STEP 4: TRAIN - TEST SPLIT
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=7
)
# STEP 5: TRAIN MODEL
model = LinearRegression()
model.fit(X_train, y_train)
print(model)
```



```
# STEP 6: PREDICTION
predictions = model.predict(X_test)
# STEP 7: EVALUATION
mse = mean_squared_error(y_test, predictions)
print("MSE:", mse)
print("Marks per Class:", model.coef_[0])
print("Base Marks:", model.intercept_)
# STEP 8: VISUALIZATION
plt.scatter(X, y)
plt.plot(X, model.predict(X))
plt.xlabel("Classes Attended")
plt.ylabel("Internal Marks")
plt.title("Classes Attended vs Internal Marks")
plt.show()
```

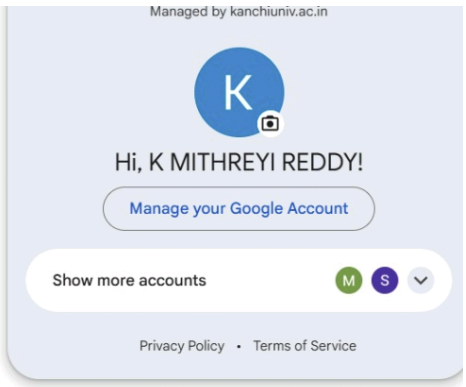
```
  Student  Classes_Attended  Internal_Marks
0      S1                30              35
1      S2                35              49
2      S3                40              38
3      S4                45              46
4      S5                50              50
5      S6                55              55
6      S7                60              60
7      S8                65              65
8      S9                70              70
9     S10                75              75
LinearRegression()
MSE: 1.700367647058822
Marks per Class: 0.8264705882352942
Base Marks: 10.92647058823529
```


Classes Attended vs Internal Marks
Internal Marks
Classes Attended
75
70
65
60
55
50
45
40
35
30
40
50
60
70

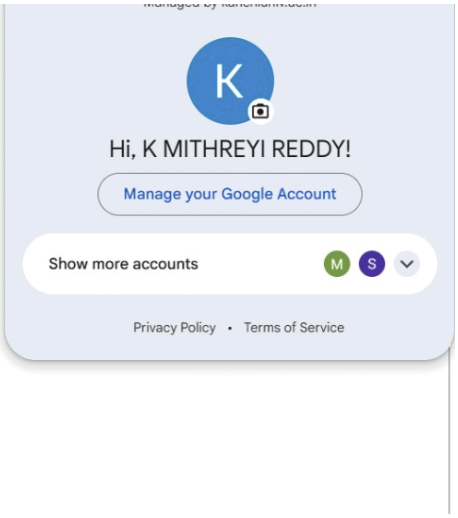

Managed by kanchiuniv.ac.in
K
Hi, K MITHREYI REDDY!
Manage your Google Account
Show more accounts
M
S
Privacy Policy • Terms of Service

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# Creating dataset
data = {
    'Country': ['India', 'USA', 'India', 'USA', 'UAE', 'India'],
    'Age': [22, 25, np.nan, 30, 28, 35],
    'Salary': [40000, 60000, 50000, np.nan, 72000, 58000],
    'Purchased': ['No', 'Yes', 'Yes', 'No', 'Yes', 'Yes']
}

# Convert to DataFrame
df = pd.DataFrame(data)

print("---- ORIGINAL RAW DATA ----")
print(df)
print("\n")

# Missing values
print("---- MISSING VALUES COUNT ----")
print(df.isnull().sum())
print("\n")

# Handling missing values
df['Age'] = df['Age'].fillna(df['Age'].mean())
df['Salary'] = df['Salary'].fillna(df['Salary'].mean())
```

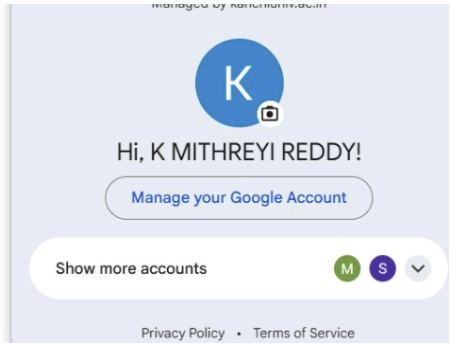
```
print("---- DATA AFTER CLEANING ----")
print(df)
print("\n")

# Encoding categorical data
df_encoded = pd.get_dummies(df, columns=['Country'], drop_first=True)

# Encoding target variable
df_encoded['Purchased'] = df_encoded['Purchased'].map({'Yes': 1, 'No': 0})

print("---- DATA AFTER ENCODING CATEGORICAL FEATURES ----")
print(df_encoded)
print("\n")

# Splitting features and target
X = df_encoded.drop('Purchased', axis=1)
y = df_encoded['Purchased']

# Feature scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("---- SCALED FEATURES ----")
print(X_scaled)
```



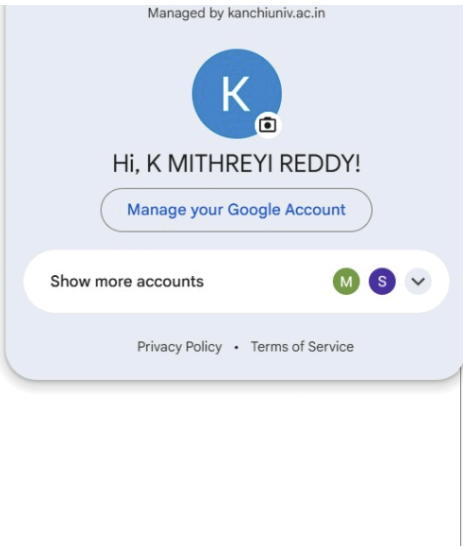
```
---- ORIGINAL RAW DATA ----
  Country   Age   Salary Purchased
0   India  22.0  40000.0        No
1     USA  25.0  60000.0       Yes
2   India   NaN  50000.0       Yes
3     USA  30.0      NaN        No
4     UAE  28.0  72000.0       Yes
5   India  35.0  58000.0       Yes


---- MISSING VALUES COUNT ----
Country      0
Age          1
Salary       1
Purchased    0
dtype: int64


---- DATA AFTER CLEANING ----
  Country   Age   Salary Purchased
0   India  22.0  40000.0        No
1     USA  25.0  60000.0       Yes
2   India  28.0  50000.0       Yes
3     USA  30.0  56000.0        No
4     UAE  28.0  72000.0       Yes
5   India  35.0  58000.0       Yes
```



```
---- DATA AFTER ENCODING CATEGORICAL FEATURES ----
    Age   Salary  Purchased  Country_UAE  Country_USA
0  22.0  40000.0          0        False        False
1  25.0  60000.0          1        False         True
2  28.0  50000.0          1        False        False
3  30.0  56000.0          0        False         True
4  28.0  72000.0          1         True        False
5  35.0  58000.0          1        False        False


---- SCALED FEATURES ----
[[-1.48461498 -1.6444529  -0.4472136  -0.70710678]
 [-0.74230749  0.41111323 -0.4472136   1.41421356]
 [ 0.         -0.61666984 -0.4472136  -0.70710678]
 [ 0.49487166  0.         -0.4472136   1.41421356]
 [ 0.          1.6444529   2.23606798 -0.70710678]
 [ 1.73205081  0.20555661 -0.4472136  -0.70710678]]
```



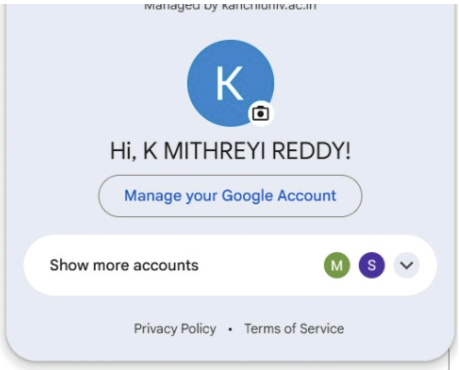
```python
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn import datasets

# Load dataset
diabetes = datasets.load_diabetes()

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    diabetes.data,
    diabetes.target,
    test_size=0.2,
    random_state=42
)

# Train models
lr = LinearRegression()
lr.fit(X_train, y_train)

ridge = Ridge(alpha=1.0)
ridge.fit(X_train, y_train)

lasso = Lasso(alpha=0.1)
lasso.fit(X_train, y_train)
```



```python
# Predictions and MSE
lr_mse = mean_squared_error(y_test, lr.predict(X_test))
ridge_mse = mean_squared_error(y_test, ridge.predict(X_test))
lasso_mse = mean_squared_error(y_test, lasso.predict(X_test))

# Print results
print(f"Linear Regression MSE: {lr_mse:.2f}")
print(f"Ridge Regression MSE: {ridge_mse:.2f}")
print(f"Lasso Regression MSE: {lasso_mse:.2f}")
```

```
Linear Regression MSE: 2900.19
Ridge Regression MSE: 3077.42
Lasso Regression MSE: 2798.19
```

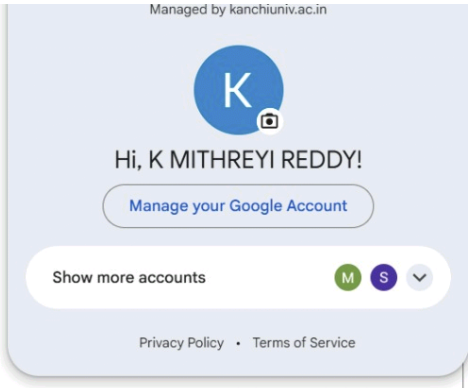
```python
# Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

# Create dataset
data = {
    'StudyHours': [5, 2, 8, 1, 6, 3, 7, 4],
    'Attendance': [80, 50, 90, 40, 85, 60, 88, 65],
    'Result': ['Pass', 'Fail', 'Pass', 'Fail', 'Pass', 'Fail', 'Pass', 'Fail']
}

# Convert to DataFrame
df = pd.DataFrame(data)

# Features and target
X = df[['StudyHours', 'Attendance']]
y = df['Result']

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

# Train model
model = GaussianNB()
model.fit(X_train, y_train)
```



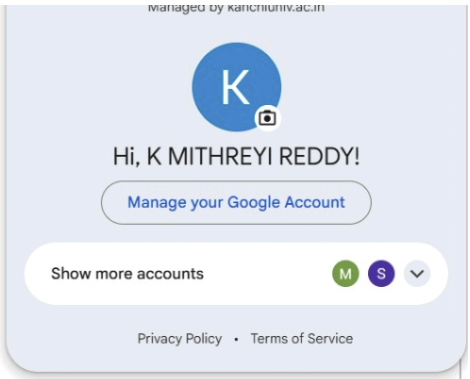
```python
# Prediction
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

# Output
print("Predicted Results:", y_pred)
print("Actual Results:", y_test.values)
print("Accuracy:", accuracy)
```

```
Predicted Results: ['Fail' 'Fail']
Actual Results: ['Fail' 'Fail']
Accuracy: 1.0
```

```python
# Import libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# 1. Load Dataset
data = load_breast_cancer()
X = data.data
y = data.target

# 2. Split Data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3. Feature Scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 4. Normal Logistic Regression (very high C = almost no regularization)
normal_model = LogisticRegression(C=1e6, solver='liblinear')
normal_model.fit(X_train_scaled, y_train)

normal_pred = normal_model.predict(X_test_scaled)
```

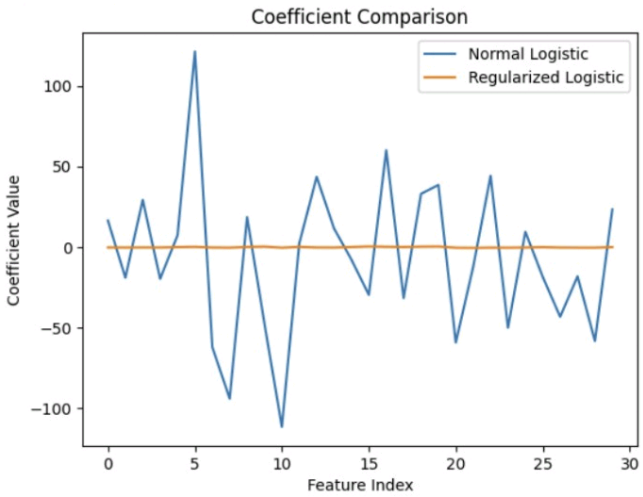


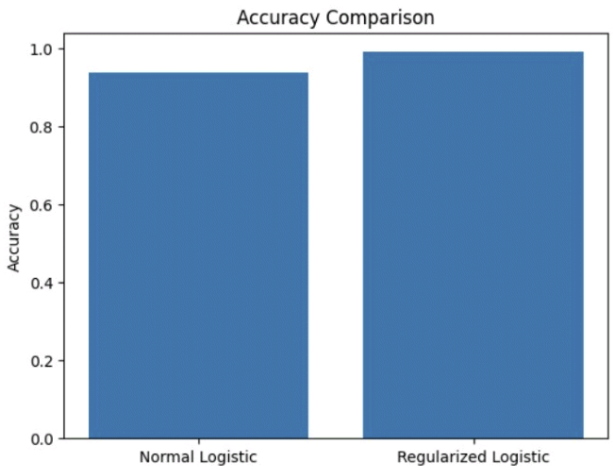
```
normal_acc = accuracy_score(y_test, normal_pred)

# 5. Regularized Logistic Regression (L2 by default)
regularized_model = LogisticRegression(C=0.1, penalty='l2', solver='liblinear')
regularized_model.fit(X_train_scaled, y_train)

reg_pred = regularized_model.predict(X_test_scaled)
reg_acc = accuracy_score(y_test, reg_pred)

# 6. Print Results
print("Normal Logistic Accuracy:", normal_acc * 100)
print("Regularized Logistic Accuracy:", reg_acc * 100)

# 7. Compare Coefficients
plt.figure()
plt.plot(normal_model.coef_.flatten(), label='Normal Logistic')
plt.plot(regularized_model.coef_.flatten(), label='Regularized Logistic')
plt.legend()
plt.title("Coefficient Comparison")
plt.xlabel("Feature Index")
plt.ylabel("Coefficient Value")
plt.show()

# 8. Accuracy Comparison Bar Graph
plt.figure()
models = ['Normal Logistic', 'Regularized Logistic']
accuracies = [normal_acc, reg_acc]

plt.bar(models, accuracies)
```

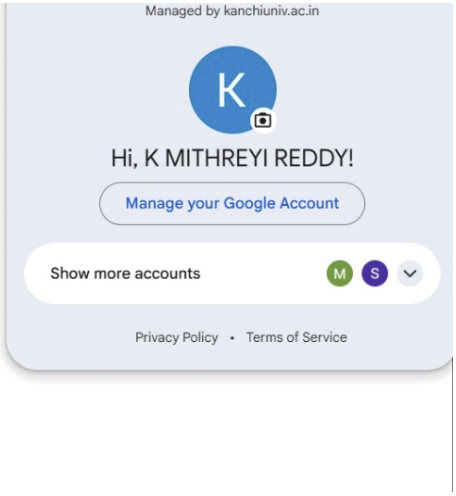


```
plt.title("Accuracy Comparison")
plt.ylabel("Accuracy")
plt.show()
```

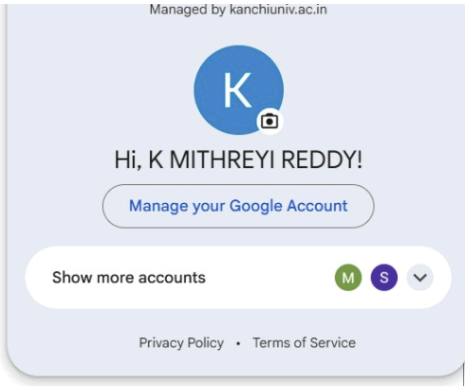
```
Normal Logistic Accuracy: 93.85964912280701
Regularized Logistic Accuracy: 99.12280701754386
```




Accuracy Comparison
Accuracy
1.0
0.8
0.6
0.4
0.2
0.0
Normal Logistic
Regularized Logistic

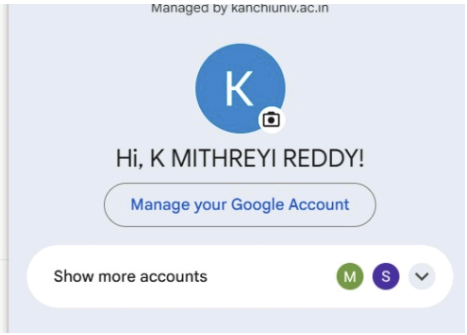

Managed by kanchiuniv.ac.in
K
Hi, K MITHREYI REDDY!
Manage your Google Account
Show more accounts
M
S
Privacy Policy • Terms of Service

```python
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, StackingClassifier
from sklearn.svm import SVC
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Load dataset
iris = load_iris()

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42
)

# Random Forest
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

# AdaBoost
ada = AdaBoostClassifier(n_estimators=100, random_state=42)
ada.fit(X_train, y_train)

# Stacking
estimators = [
    ('rf', RandomForestClassifier(n_estimators=100, random_state=42)),
    ('svc', SVC(probability=True))
]
```



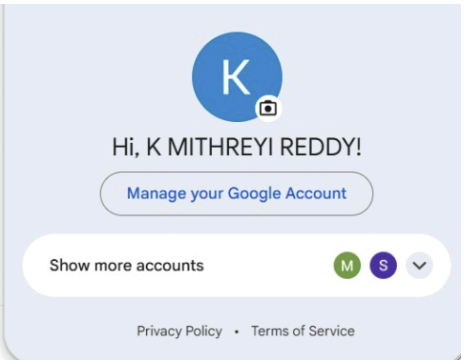
```
stack = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression()
)

stack.fit(X_train, y_train)

# Predictions and accuracy
print(f"Random Forest Accuracy: {accuracy_score(y_test, rf.predict(X_test)):.2f}")
print(f"AdaBoost Accuracy: {accuracy_score(y_test, ada.predict(X_test)):.2f}")
print(f"Stacking Accuracy: {accuracy_score(y_test, stack.predict(X_test)):.2f}")
```

```
Random Forest Accuracy: 1.00
AdaBoost Accuracy: 0.93
Stacking Accuracy: 1.00
```

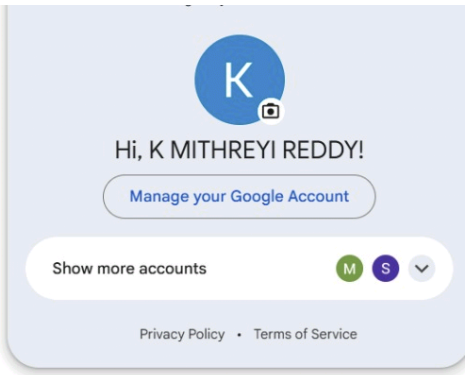


```python
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
iris = datasets.load_iris()
X_train, X_test, y_train, y_test = train_test_split(
iris.data, iris.target, test_size=0.2, random_state=42
)
dt_clf = DecisionTreeClassifier(max_depth=3)
dt_clf.fit(X_train, y_train)
y_pred = dt_clf.predict(X_test)
print(f"Decision Tree Classifier Accuracy: {accuracy_score(y_test, y_pred):.2f}")
print("Feature Importances:", dt_clf.feature_importances_)
```

```
Decision Tree Classifier Accuracy: 1.00
Feature Importances: [0.         0.         0.93462632 0.06537368]
```



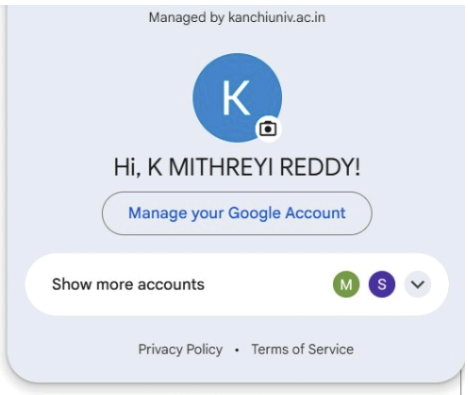
```python
# Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris
# Load dataset
data = load_iris()
X = data.data
y = data.target
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
# Linear Kernel
model_linear = SVC(kernel='linear')
model_linear.fit(X_train, y_train)
pred_linear = model_linear.predict(X_test)
acc_linear = accuracy_score(y_test, pred_linear)
print("Linear Kernel Accuracy:", acc_linear)
# Polynomial Kernel
model_poly = SVC(kernel='poly', degree=3)
model_poly.fit(X_train, y_train)
pred_poly = model_poly.predict(X_test)
acc_poly = accuracy_score(y_test, pred_poly)
print("Polynomial Kernel Accuracy:", acc_poly)
```



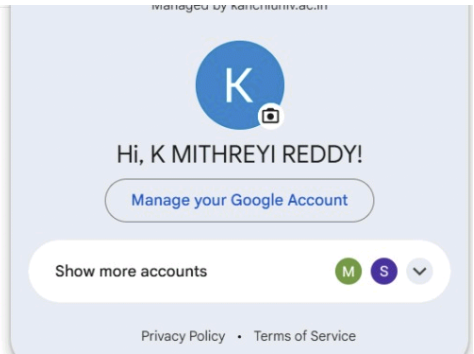
```
# RBF Kernel
model_rbf = SVC(kernel='rbf')
model_rbf.fit(X_train, y_train)
pred_rbf = model_rbf.predict(X_test)
acc_rbf = accuracy_score(y_test, pred_rbf)
print("RBF Kernel Accuracy:", acc_rbf)
```

```
Linear Kernel Accuracy: 1.0
Polynomial Kernel Accuracy: 1.0
RBF Kernel Accuracy: 1.0
```

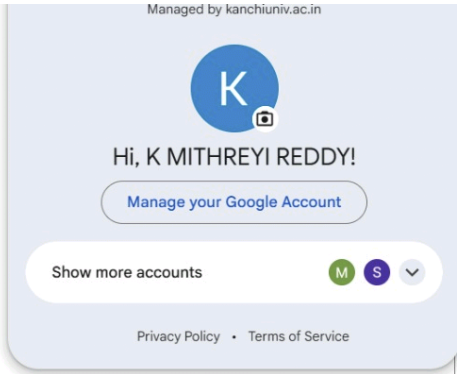
```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
# Step 2: Load dataset
data = pd.read_csv("mobile_price.csv")
# Step 3: Define features and target
X = data.drop("price_range", axis=1)
y = data["price_range"]
# Step 4: Train-test split
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42
)
# Step 5: Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
# Step 6: Create KNN model
knn = KNeighborsClassifier(n_neighbors=5)
# Step 7: Train model
knn.fit(X_train, y_train)
# Step 8: Prediction
y_pred = knn.predict(X_test)
# Step 9: Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```



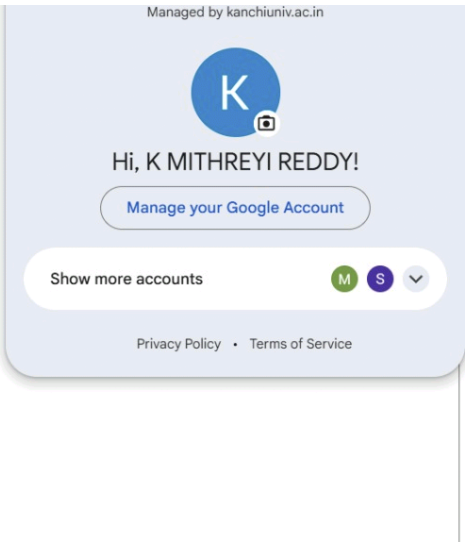
```
Accuracy: 0.0
Confusion Matrix:
 [[0 3 0 0]
 [1 0 1 0]
 [1 1 0 0]
 [0 1 2 0]]
Classification Report:
              precision    recall  f1-score   support

           0       0.00      0.00      0.00       3.0
           1       0.00      0.00      0.00       2.0
           2       0.00      0.00      0.00       2.0
           3       0.00      0.00      0.00       3.0

    accuracy                           0.00      10.0
   macro avg       0.00      0.00      0.00      10.0
weighted avg       0.00      0.00      0.00      10.0
```



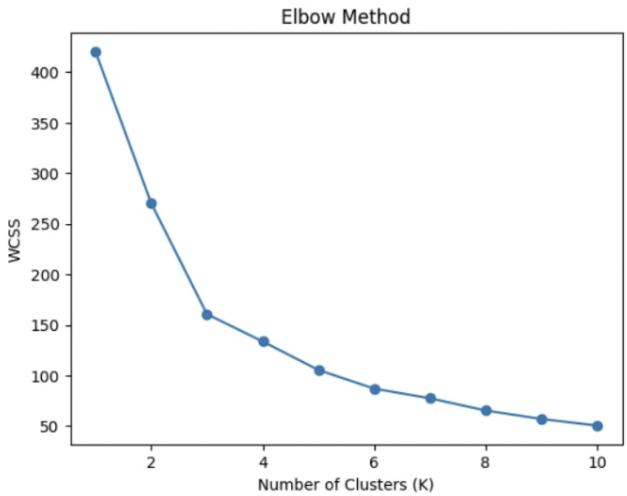
```python
# Step 1: Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
import numpy as np # Added for dummy data generation

# Step 2: Load dataset
try:
    data = pd.read_csv("Mall_Customers.csv")
except FileNotFoundError:
    print("Mall_Customers.csv not found. Generating dummy data for demonstration.")
    # Generate a synthetic dataset for demonstration
    np.random.seed(42)
    data = pd.DataFrame({
        'Age': np.random.randint(18, 70, size=200),
        'Annual Income (k$)': np.random.randint(15, 120, size=200),
        'Spending Score (1-100)': np.random.randint(1, 100, size=200)
    })
    print("Dummy data generated.")

# Step 3: Select numerical features
X = data[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']]
# Step 4: Feature Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

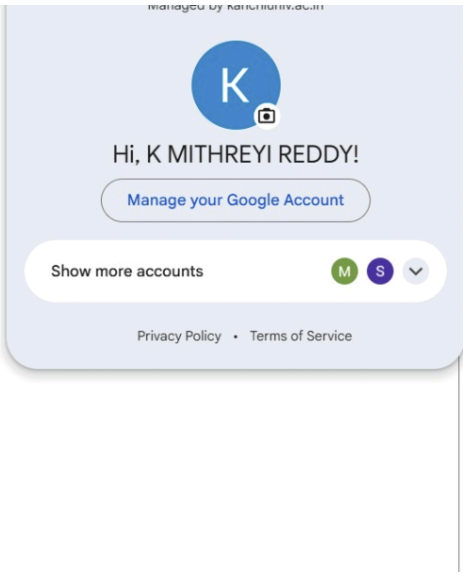


```python
# Step 5: Apply PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
print("Explained Variance Ratio:", pca.explained_variance_ratio_)
# Step 6: Elbow Method
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, random_state=0, n_init='auto')
    kmeans.fit(X_pca)
    wcss.append(kmeans.inertia_)
# Step 7: Plot Elbow Graph
plt.plot(range(1, 11), wcss, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.show()
# Step 8: Apply KMeans with optimal K (e.g., K=5)
kmeans = KMeans(n_clusters=5, random_state=0, n_init='auto')
clusters = kmeans.fit_predict(X_pca)
# Step 9: Plot Clusters
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters, cmap='viridis')
plt.title("K-Means Clustering with PCA")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.show()
```

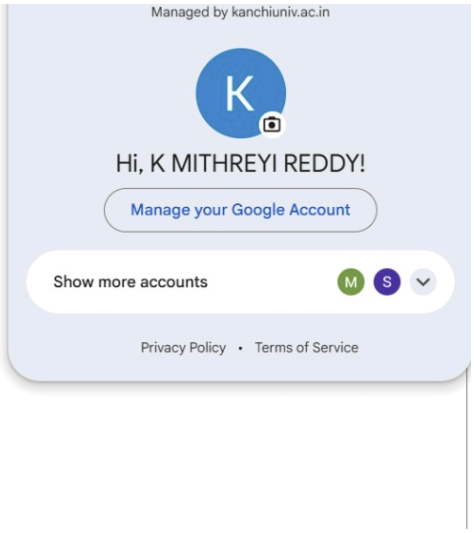


```
Mall_Customers.csv not found. Generating dummy data for demonstration.
Dummy data generated.
Explained Variance Ratio: [0.35221078 0.34845691]
```






K-Means Clustering with PCA
PCA Component 2
PCA Component 1
2
1
0
−1
−2
−2
−1
0
1
2


Managed by kanchiuniv.ac.in
K
Hi, K MITHREYI REDDY!
Manage your Google Account
Show more accounts
M
S
Privacy Policy • Terms of Service